

Mudanças no Projeto para Implementação de HA

Visão Geral das Alterações

Este documento detalha todas as mudanças implementadas no projeto original para transformá-lo em um cluster PostgreSQL de alta disponibilidade funcional. As alterações incluem novos arquivos de configuração, modificações em arquivos existentes, novos scripts de automação e mudanças na arquitetura Docker.

1. Arquivos de Configuração PostgreSQL

1.1 Arquivo: conf/postgresql-primary.conf

Propósito: Configuração específica para o servidor primário do cluster

Conteúdo Principal:

```
# Configurações Básicas
listen_addresses = '*'
port = 5432
max_connections = 100

# Configurações de Performance
shared_buffers = 128MB
effective_cache_size = 512MB
wal_buffers = 16MB

# Configurações de Replicação
wal_level = replica
max_wal_senders = 10
hot_standby = on
archive_mode = on

# Repmgr
shared_preload_libraries = 'repmgr'
```

Propósito:

- **wal_level = replica:** Habilita geração de logs WAL necessários para replicação
- **max_wal_senders = 10:** Permite até 10 conexões de replicação simultâneas
- **hot_standby = on:** Permite que secundários aceitem conexões de leitura
- **shared_preload_libraries = 'repmgr':** Carrega a extensão repmgr no startup
- **Configurações de performance otimizadas** para servidor primário

1.2 Arquivo: conf/postgresql-standby.conf

Propósito: Configuração específica para o servidor secundário

Conteúdo Principal:

```
# Configurações de Recuperação
recovery_target_timeline = latest
primary_conninfo = 'host=postgresql-primary port=5432 user=repmgr password=repmgr123'
```

Propósito:

- **recovery_target_timeline = latest:** Garante que o standby sempre siga a timeline mais recente
- **primary_conninfo:** Define como o standby se conecta ao primário
- **Configurações idênticas ao primário** para performance e replicação

1.3 Arquivo: conf/postgresql-witness.conf

Propósito: Configuração para o nó testemunha (quorum)

Conteúdo Principal:

```
# Configurações de Recuperação
recovery_target_timeline = latest
primary_conninfo = 'host=postgresql-primary port=5432 user=repmgr password=repmgr123'
```

Propósito:

- **Configuração similar ao standby** mas para nó de quorum
- **Não armazena dados de aplicação** mas participa de decisões de failover
- **Previne cenários de split-brain** em clusters com múltiplos nós

2. Configuração de Autenticação

2.1 Arquivo: conf/pg_hba.conf

Mudanças Implementadas:

Configuração para rede Docker

```
host    all             all             172.18.0.0/16      trust
host    all             all             172.18.0.5/32     trust
host    all             all             172.18.0.6/32     trust
host    all             all             172.18.0.3/32     trust
```

Propósito:

- **Adicionadas entradas para rede Docker** (172.18.0.0/16)
- **IPs específicos para componentes** (HAProxy, Repmgr Manager, Standby)
- **Método trust** para facilitar desenvolvimento (não recomendado para produção)
- **Permite conexões de replicação** entre todos os nós do cluster

3. Configuração Repmgr

3.1 Arquivo: conf/repmgr-primary.conf

Propósito: Configuração do Repmgr para o nó primário

Conteúdo Principal:

```
node_id=1
node_name='postgresql-primary'
conninfo='host=localhost user=repmgr password=repmgr123 dbname=repmgr'
data_directory='/var/lib/postgresql/data'
```

Parâmetros::

- **node_id=1:** Identificador único do nó primário
- **node_name:** Nome do nó para identificação no cluster
- **conninfo:** String de conexão para o próprio nó
- **data_directory:** Caminho para os dados PostgreSQL

3.2 Arquivo: conf/repmgr-standby.conf

Propósito: Configuração do Repmgr para o nó secundário

Conteúdo Principal:

```
node_id=2
node_name='postgresql-standby'
conninfo='host=localhost user=repmgr password=repmgr123 dbname=repmgr'
data_directory='/var/lib/postgresql/data'
upstream_node_conninfo='host=postgresql-primary user=repmgr password=repmgr123 dbname=repmgr'
upstream_node_id=1
```

Parâmetros::

- **node_id=2:** Identificador único do nó secundário
- **upstream_node_conninfo:** Como conectar ao nó primário
- **upstream_node_id=1:** Referência ao ID do nó primário
- **Configuração para replicação** e failover

3.3 Arquivo: conf/repmgr-witness.conf

Propósito: Configuração do Repmgr para o nó testemunha

Conteúdo Principal:

```
node_id=3
node_name='postgresql-witness'
conninfo='host=localhost user=repmgr password=repmgr123 dbname=repmgr'
data_directory='/var/lib/postgresql/data'
upstream_node_conninfo='host=postgresql-primary user=repmgr password=repmgr123 dbname=repmgr'
```

```
upstream_node_id=1
```

Parâmetros::

- **node_id=3**: Identificador único do nó testemunha
- **Configuração para quorum** e decisões de failover
- **Não armazena dados de aplicação** mas participa de decisões

3.4 Arquivo: conf/repmgr-manager.conf

Propósito: Configuração do Repmgr para o container de gerenciamento

Conteúdo Principal:

```
node_id=1
node_name='repmgr-manager'
conninfo='host=postgresql-primary user=repmgr password=repmgr123 dbname=repmgr'
log_file='/var/log/repmgr/repmgr.log'
log_level=INFO
data_directory='/var/lib/postgresql/data'
```

Detalhamento:

- **Configuração para container de gerenciamento**
- **Logging configurado** para monitoramento
- **Conecta ao primário** para operações administrativas

4. Configuração HAProxy

4.1 Arquivo: conf/haproxy.cfg

Propósito: Configuração do balanceador de carga HAProxy

Conteúdo Principal:

```
global
    log stdout format raw local0 info

defaults
    mode tcp
    timeout connect 5s
    timeout client 30s
    timeout server 30s

# Página de Estatísticas
listen stats
    bind *:8404
    mode http
    stats enable
    stats uri /stats
    stats refresh 10s
    stats auth admin:admin123

# Cluster PostgreSQL
listen postgres_cluster
    bind *:5000
    mode tcp
    option tcp-check
    balance roundrobin
    server postgresql-primary postgresql-primary:5432 check port 5432 maxconn 100 weight 100
    server postgresql-standby postgresql-standby:5432 check port 5432 maxconn 100 weight 50
```

Parâmetros::

- **mode tcp**: HAProxy atua como proxy TCP para PostgreSQL
- **balance roundrobin**: Distribui conexões alternadamente
- **weight 100/50**: Primário recebe 2x mais conexões que standby
- **option tcp-check**: Verifica se PostgreSQL está respondendo
- **Página de estatísticas** para monitoramento (porta 8404)

5. Dockerfiles

5.1 Arquivo: Dockerfile.postgresql

Propósito: Dockerfile customizado para containers PostgreSQL com Repmgr

Conteúdo Principal:

```
FROM postgres:17
```

```
# Install repmgr
```

```
RUN apt-get update && apt-get install -y \
    postgresql-17-repmgr \
    && rm -rf /var/lib/apt/lists/*
```

```
# Create repmgr user and database
```

```
RUN echo "CREATE USER repmgr WITH SUPERUSER PASSWORD 'repmgr123';" > /docker-entrypoint-initdb.d/01-
    create-repmgr-user.sql
```

```
RUN echo "CREATE DATABASE repmgr OWNER repmgr;" >> /docker-entrypoint-initdb.d/01-create-repmgr-
    user.sql
```

```
# Create log directory
```

```
RUN mkdir -p /var/log/repmgr
```

```
# Set proper permissions
```

```
RUN chown -R postgres:postgres /var/log/repmgr
```

Detalhamento:

- **Baseado em PostgreSQL 17:** Versão mais recente e estável
- **Instalação do Repmgr:** Pacote postgresql-17-repmgr
- **Criação do usuário repmgr:** Com privilégios SUPERUSER
- **Criação do banco repmgr:** Para metadados do cluster
- **Diretório de logs:** Para monitoramento do Repmgr
- **Permissões corretas:** Para funcionamento adequado

5.2 Arquivo: Dockerfile.repmgr

Propósito: Dockerfile para container de gerenciamento Repmgr

Conteúdo Principal:

```
FROM postgres:17
```

```
# Install repmgr
```

```
RUN apt-get update && apt-get install -y \
    postgresql-17-repmgr \
    && rm -rf /var/lib/apt/lists/*
```

```
# Create repmgr user and database
```

```
RUN echo "CREATE USER repmgr WITH SUPERUSER PASSWORD 'repmgr123';" > /docker-entrypoint-initdb.d/01-
    create-repmgr-user.sql
```

```
RUN echo "CREATE DATABASE repmgr OWNER repmgr;" >> /docker-entrypoint-initdb.d/01-create-repmgr-
    user.sql
```

```
# Create log directory
```

```
RUN mkdir -p /var/log/repmgr
```

```
# Set proper permissions
```

```
RUN chown -R postgres:postgres /var/log/repmgr
```

Propósito:

- **Similar ao Dockerfile.postgresql:** Mas para container de gerenciamento
- **Foco em operações administrativas:** Não armazena dados de aplicação
- **Configuração para monitoramento:** Logs e metadados do cluster

6. Docker Compose

6.1 Arquivo: docker-compose.yml

Principais Mudanças:

6.1.1 Estrutura de Serviços

```
services:
  postgresql-primary:
    build:
      context: .
      dockerfile: Dockerfile.postgresql
    container_name: postgresql-primary
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres123
      POSTGRES_DB: postgres
    volumes:
      - pg_primary_data:/var/lib/postgresql/data
```

```

- ./conf/postgresql-primary.conf:/etc/postgresql/postgresql.conf
- ./conf/pg_hba.conf:/etc/postgresql/pg_hba.conf
- ./conf/repmgr-primary.conf:/etc/repmgr.conf
expose:
- 5432
ports:
- 5432:5432
command: ["postgres", "-c", "config_file=/etc/postgresql/postgresql.conf"]
networks:
  postgres_network:
    ipv4_address: 172.18.0.2

```

Propósito:

- **Build customizado:** Usa Dockerfile.postgresql em vez de imagem padrão
- **Volumes montados:** Configurações específicas para cada nó
- **IPs fixos:** Garantem conectividade estável
- **Comando customizado:** Usa arquivo de configuração específico

6.1.2 Rede Personalizada

```

networks:
  postgres_network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.18.0.0/16

```

Propósito:

- **Rede dedicada:** Isolamento completo do cluster
- **Subnet definida:** 172.18.0.0/16 para todos os componentes
- **IPs fixos:** Evitam problemas de conectividade

6.1.3 Serviços Adicionados

- **postgresql-standby:** Servidor secundário
- **postgresql-witness:** Nó de quorum
- **haproxy:** Balanceador de carga
- **repmgr-manager:** Container de gerenciamento
- **pgadmin:** Interface web de administração

7. Scripts de Automação

7.1 Arquivo: start_ha.sh

Propósito: Ponto de entrada principal para iniciar o cluster

Conteúdo Principal:

```

#!/bin/bash

echo "Starting PostgreSQL HA Cluster..."
echo

# Make the auto setup script executable
chmod +x scripts/auto_setup.sh

# Run the auto setup
./scripts/auto_setup.sh

echo
echo "PostgreSQL HA Cluster is ready!"
echo
echo "Quick access:"
echo "  • HAProxy Stats: http://localhost:8404 (admin/admin123)"
echo "  • PgAdmin: http://localhost:8080 (admin@admin.com/admin123)"
echo "  • PostgreSQL: localhost:5432 (primary), localhost:5433 (standby)"
echo "  • HAProxy Load Balancer: localhost:5000"
echo
echo "Test failover:"
echo "  docker compose stop postgresql-primary"
echo "  docker compose start postgresql-primary"

```

Propósito:

- **Script principal:** Ponto único de entrada para iniciar o cluster
- **Execução automática:** Chama o script de setup completo

- **Informações de acesso:** Mostra todos os pontos de entrada
- **Instruções de teste:** Comandos para testar failover

7.2 Arquivo: scripts/auto_setup.sh

Propósito: Script completo de configuração automática do cluster

Principais Etapas:

7.2.1 Limpeza e Inicialização

```
# Step 1: Clean up any existing setup
docker compose down -v 2>/dev/null || true
docker volume prune -f 2>/dev/null || true

# Step 2: Start all services
docker compose up -d
```

Propósito: Remove configurações anteriores e inicia containers limpos

7.2.2 Preparação do Banco

```
# Step 4: Create test database and table
docker exec postgresql-primary su postgres -c "psql -c \"CREATE DATABASE testdb;\" 2>/dev/null || echo 'Database testdb already exists'"
docker exec postgresql-primary su postgres -c "psql -d testdb -c \"CREATE TABLE IF NOT EXISTS test_table (id SERIAL PRIMARY KEY, name VARCHAR(100), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);\""
```

Propósito: Cria banco e tabela de teste para validação

7.2.3 Registro do Primário

```
# Step 5: Register primary with repmgr
docker exec postgresql-primary su postgres -c "repmgr primary register --force"
```

Propósito: Registra nó primário no sistema Repmgr

7.2.4 Backup e Configuração do Standby

```
# Step 8: Create base backup
docker exec postgresql-primary su postgres -c "pg_basebackup -h localhost -U repmgr -D /tmp/standby_backup -v -P -W"

# Step 9: Copy backup to standby
docker cp postgresql-primary:/tmp/standby_backup ./standby_backup_temp
docker cp ./standby_backup_temp/. postgresql-standby:/var/lib/postgresql/data/
```

Propósito: Cria backup físico completo e copia para standby

7.2.5 Configuração de Replicação

```
# Step 10: Configure standby for replication
docker exec postgresql-standby su postgres -c "touch /var/lib/postgresql/data/standby.signal"
docker exec postgresql-standby su postgres -c "echo \"primary_conninfo = 'host=postgresql-primary port=5432 user=repmgr password=repmgr123 application_name=postgresql-standby'\" >> /var/lib/postgresql/data/postgresql.conf"
```

Propósito: Configura standby para replicação com PostgreSQL 12+

7.2.6 Registro do Standby

```
# Step 14: Register standby with repmgr
docker exec postgresql-primary su postgres -c "psql -d repmgr -c \"INSERT INTO repmgr.nodes (node_id, node_name, type, location, priority, conninfo, repluser, config_file, upstream_node_id, slot_name) VALUES (2, 'postgresql-standby', 'standby', 'default', 50, 'host=postgresql-standby user=repmgr password=repmgr123 dbname=repmgr', 'repmgr', '/etc/repmgr.conf', 1, 'repmgr_2_postgresql_standby') ON CONFLICT (node_id) DO NOTHING;\""
```

Propósito: Registra standby no sistema Repmgr

7.2.7 Teste de Validação

```
# Step 15: Test replication
docker exec postgresql-primary su postgres -c "psql -d testdb -c \"INSERT INTO test_table (name) VALUES ('Auto Setup Test - $(date)');\""
sleep 5
```

```
docker exec postgresql-standby su postgres -c "psql -d testdb -c \"SELECT COUNT(*) as total_records
FROM test_table;\""
```

Propósito: Valida que a replicação está funcionando

8. Estrutura Final do Projeto

8.1 Arquivos Principais

```
dba_postgresql/
├── README.md                # Documentação prática
├── CONCEITOS_DETALHADOS.md  # Documentação técnica completa
├── MUDANCAS_IMPLEMENTADAS.md # Este arquivo
├── start_ha.sh              # Script principal
├── docker-compose.yml       # Orquestração completa
├── scripts/
│   └── auto_setup.sh        # Setup automático
├── conf/
│   ├── postgresql-primary.conf # Config primário
│   ├── postgresql-standby.conf # Config standby
│   ├── postgresql-witness.conf # Config witness
│   ├── pg_hba.conf           # Autenticação
│   ├── repmgr-primary.conf    # Repmgr primário
│   ├── repmgr-standby.conf    # Repmgr standby
│   ├── repmgr-witness.conf    # Repmgr witness
│   ├── repmgr-manager.conf    # Repmgr manager
│   └── haproxy.cfg            # Balanceador de carga
├── Dockerfile.postgresql    # Container PostgreSQL
├── Dockerfile.repmgr        # Container Repmgr
└── Dockerfile.pgadmin       # Container PgAdmin
```

8.2 Componentes do Cluster

- **postgresql-primary:** Servidor primário (porta 5432)
- **postgresql-standby:** Servidor secundário (porta 5433)
- **postgresql-witness:** Nó de quorum (porta 5434)
- **haproxy:** Balanceador de carga (porta 5000/8404)
- **repmgr-manager:** Gerenciamento do cluster
- **pgadmin:** Interface web (porta 8080)

9. Resumo das Implementações

9.1 Arquitetura Implementada

- **Cluster de 3 nós:** Primário, secundário e testemunha
- **Balanceamento de carga:** HAProxy distribuindo conexões
- **Replicação assíncrona:** Performance otimizada
- **IPs fixos:** Conectividade estável
- **Setup automatizado:** Um comando para iniciar tudo

9.2 Funcionalidades Implementadas

- **Replicação PostgreSQL:** Funcionando com WAL
- **Load Balancing:** HAProxy distribuindo carga
- **Failover:** Promoção manual de standby
- **Monitoramento:** HAProxy stats e PgAdmin
- **Automação:** Scripts para setup completo
- **Documentação:** Completa e detalhada

9.3 Pontos de Acesso

- **HAProxy Load Balancer:** localhost:5000
- **HAProxy Stats:** <http://localhost:8404>
- **PgAdmin:** <http://localhost:8080>
- **PostgreSQL Primário:** localhost:5432
- **PostgreSQL Standby:** localhost:5433

9.4 Próximos Passos

- **Segurança:** Implementar autenticação segura
- **Monitoramento:** Prometheus + Grafana
- **Backup:** WAL archiving e PITR
- **Failover automático:** Repmgrd

- **Performance:** Tuning e connection pooling